ApplyWizz DicePilot

Document 02 - TRD - Technical Requirements Document

Technical architecture and non-negotiable engineering decisions for V1

Product	ApplyWizz DicePilot
Version	V1.0 - Pilot Specification
Scope	V1 only: one internal test candidate, up to 20 Dice C2C + Easy Apply jobs, sequential processing
Date	20 August 2026

1. Technical Objective

Implement DicePilot as an extension of the existing ApplyWizz scraping codebase, preserving current Indeed/Supabase capabilities while adding a clean Dice-specific discovery and application-execution module. V1 must prioritize deterministic execution, auditability, safe candidate-data usage, and a clean path to later multi-candidate scaling.

2. Existing Assets to Reuse

Existing asset	V1 use
ApplyWizz scraper repository	Primary codebase; Dice is added as isolated modules rather than replacing existing Indeed functionality.
Supabase integration	Reuse provider and patterns; move secrets to environment variables and add Dice/application tables.
Role taxonomy / karmafy_jobrole.csv	Reuse role names, alternate roles, and keywords where applicable.
Existing job filtering/dedupe patterns	Reuse principles for normalized URLs, role matching, and database-level uniqueness.
ApplyWizz candidate-details route	Authoritative runtime source for candidate profile/application facts and resume location.
Existing Flask operator UI	May be extended minimally for V1 monitoring; not the execution runtime.

3. Recommended V1 Stack

Layer	Decision	Notes
Language	Python 3.11+	Matches existing scraper ecosystem and Dice reference code.
Dice browser automation	Playwright for Python	Preferred over Selenium for persistent contexts, deterministic locators, file uploads, tracing, and future scaling.
Database	PostgreSQL via Supabase	Business state and idempotency source of truth.
Operator backend	Existing Flask app for internal monitoring	Keep UI lightweight; browser workers must not run as long-lived Vercel request threads.
Worker runtime	Dedicated Python process/container	Owns persistent browser context and sequential application execution.
Candidate data	Existing ApplyWizz API route	Normalize through candidate_adapter.py.
File retrieval	HTTP/S3 fetch through controlled backend helper	Resume is downloaded to a temporary candidate-specific working path when needed.
Logging	Structured Python logging + application_events table	No status-only local text files as source of truth.
Testing	pytest + Playwright mocks/fixtures where possible	Unit tests for classifiers/mappers; integration tests for state transitions.

4. Dice Module Structure

Required isolation
Do not mix Dice-specific selectors, C2C rules, or Easy Apply browser actions into existing Indeed scraper files. The Dice package must be independently testable and replaceable.

Module	Responsibility
dice/scraper.py	Dice search/listing acquisition for V1 search inputs.
dice/job_parser.py	Normalize Dice job ID, title, company, location, employment type, description, canonical URL.
dice/c2c_classifier.py	Classify C2C status from structured employment fields + description evidence.
dice/easy_apply_detector.py	Detect positive Dice Easy Apply eligibility.
dice/candidate_adapter.py	Map existing ApplyWizz candidate payload to normalized application facts.
dice/browser.py	Create/reuse one persistent candidate Dice browser context; session health checks.
dice/form_questions.py	Interpret supported form fields and map them to normalized candidate facts.
dice/easy_apply.py	Deterministic Easy Apply state machine and resume handling.
dice/submission_verifier.py	Positive verification of successful Dice application state.
dice/worker.py	Claim next queued application, enforce candidate lock, run state machine, persist events/outcomes.
db/supabase_client.py	Shared Supabase initialization from environment variables.
db/application_repository.py	Atomic queue claims, status changes, events, idempotency, retry bookkeeping.

5. System Architecture

The production V1 flow must separate job discovery from application execution. Search/discovery can be lightweight; logged-in application execution is browser-based and stateful.

1. Dice discovery obtains candidate-relevant job URLs and structured job metadata.
2. Job parser stores normalized job identity and full description.
3. C2C classifier determines CONFIRMED / LIKELY / NOT_C2C / UNKNOWN.
4. Easy Apply detector determines whether the job is eligible for V1 execution.
5. Eligible jobs are persisted and candidate-specific applications are queued in Supabase.
6. Dedicated worker acquires one candidate lock and selects the next QUEUED application.
7. Candidate adapter calls existing ApplyWizz route and normalizes candidate facts.
8. Persistent Playwright context opens the job and executes the Dice Easy Apply state machine.
9. Events and intermediate states are written to Supabase.
10. Submission verifier confirms success before SUBMITTED is written.
11. Worker releases job state and moves sequentially to the next queue item.

6. Browser and Session Requirements

Requirement	Decision
Profile model	One persistent browser profile for the V1 candidate.
Concurrency	At most one active application flow for that candidate/profile.
Session health	Before each application, verify candidate is still authenticated; if not, transition to NEEDS_INPUT or controlled login path.
Security challenge	Pause for legitimate user action. No CAPTCHA/fingerprint/security-control bypass logic.
Headless/headful	Support developer-visible/headful during V1 debugging; runtime may be headless only after reliability is demonstrated.
Artifacts	Capture screenshot/trace on errors and around final submission where feasible.
Navigation	Use explicit waits/state checks, not long fixed sleeps as the primary synchronization mechanism.

7. Candidate Adapter Rules

The candidate adapter must resolve overlapping fields into one normalized, typed model. Browser code consumes only the normalized model, never the raw JSON directly.

Normalized field	Preferred source / rule
candidate_id	client.id
full_name	client.full_name
contact_email	Defined product policy; default to intended application email from candidate profile, not guessed.
phone	additional_information.primary_phone then explicitly approved fallback; null if unusable.
visa_type	client.visa_type
eligible_to_work_in_us	additional_information.eligible_to_work_in_us
requires_future_sponsorship	additional_information.require_future_sponsorship, then client.sponsorship as fallback
willing_to_relocate	additional_information.willing_to_relocate
experience_years	additional_information.experience
desired_start_date	additional_information.desired_start_date
salary_range	client.salary_range
resume_url	additional_information.resume_url
linkedin_url	additional_information.linked_in_url

github_url	additional_information.github_url
excluded_companies	Parse additional_information.exclude_companies into normalized list

Conflict rule
Null and conflicting values must not be silently reconciled by an LLM. Use explicit precedence rules and emit a validation warning when critical fields disagree.

8. Form Question Policy

Question type	Behavior
Direct known fact	Map deterministically from normalized candidate data.
Normalized yes/no variant	Use a tested question mapping rule.
Numeric known fact	Use candidate fact only when unit/meaning clearly matches.
Job-specific skill claim	Do not infer from role name alone; if not represented in verified source data, NEEDS_INPUT.
Sensitive/legal attestation	Use explicit verified candidate answer only; otherwise NEEDS_INPUT.
Unknown free text	NEEDS_INPUT in V1; AI drafting without verified facts is not a V1 requirement.

9. C2C Classification Requirements

Contract or Third Party is a search funnel, not proof of C2C. Classification must inspect explicit description evidence.

Class	Definition
CONFIRMED	Description explicitly allows C2C / Corp-to-Corp / equivalent.
LIKELY	Structured employment type and language indicate third-party contracting but explicit C2C phrase is absent.
NOT_C2C	Description explicitly says no C2C, W2 only, no third parties, direct hire only, or equivalent exclusion.
UNKNOWN	Insufficient or conflicting evidence.

10. Easy Apply Detection Requirements

12. Prefer a Dice search-level Easy Apply filter when available.
13. Before queueing or before execution, positively inspect the job detail page for the Dice Easy Apply control.
14. Do not classify an external redirect as Dice Easy Apply.
15. Persist detector result and detector evidence/version so selector changes are diagnosable.
16. If detector is uncertain, do not auto-submit in V1.

11. State Machine

The browser implementation must be explicit and deterministic. Recommended browser states:

State	Expected action / transition
OPEN_JOB	Navigate to canonical Dice job URL and confirm expected page.
CHECK_LOGGED_IN	Confirm candidate session is authenticated.
CHECK_ELIGIBILITY	Reconfirm Easy Apply and job is not already applied.
CLICK_EASY_APPLY	Open application flow.
DISCOVER_FORM	Identify current step, supported controls, and resume state.
HANDLE_RESUME	Keep/select/replace/upload candidate resume as required.
ANSWER_QUESTIONS	Map supported questions; unknown -> NEEDS_INPUT.
NEXT_OR_REVIEW	Advance only when required fields are satisfied.
SUBMITTING	Initiate final submit and prevent duplicate click/retry.
VERIFY_SUCCESS	Observe positive success evidence.
DONE	Write SUBMITTED and final event.

12. Environment Variables

Variable	Purpose
SUPABASE_URL	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	Server-side database access; never expose in browser UI
APPLYWIZZ_API_BASE_URL	Existing candidate-details service base URL
APPLYWIZZ_API_TOKEN	Service authentication if required
DICE_BROWSER_PROFILE_ROOT	Root folder/volume for persistent candidate browser state
DICE_HEADLESS	V1 runtime mode flag
DICE_PILOT_CANDIDATE_ID	Optional pilot-only default, not a final multi-tenant design
LOG_LEVEL	Structured logging level

13. Security and Compliance Constraints

- Do not commit credentials, tokens, candidate private data, browser storage state, or downloaded resumes to Git.
- Use environment variables/secrets for Supabase and service credentials.
- Use only the candidate's authorized account/session for applications.
- Do not implement CAPTCHA, device-trust, fingerprinting, rate-limit, or other security-control bypasses.
- Do not invent resume content or candidate facts.
- Do not log sensitive candidate payloads wholesale; log field names/statuses rather than raw sensitive values.

- Keep browser profile storage access restricted to the worker runtime.

14. Technical Non-Goals for V1

- No Kubernetes/autoscaling architecture.
- No distributed multi-candidate scheduler.
- No generic multi-board browser abstraction.
- No LLM browser agent controlling every click.
- No recruiter inbox service.
- No mobile push or Telegram service.
- No resume-generation pipeline.

15. Technical Done Criteria

17. Existing Indeed functionality remains operational after Dice modules are introduced.
18. All secrets used by new Dice code are environment-driven.
19. Dice discovery can persist normalized jobs with C2C and Easy Apply evidence.
20. One pilot candidate can reuse a persistent authenticated browser profile.
21. Application worker is sequential and idempotent.
22. Known candidate questions are filled from normalized verified data only.
23. Unknown questions stop safely.
24. Submission status is verified and persisted.
25. Unit tests cover classifier, candidate adapter, and state/repository rules; integration test covers a controlled application path.